American National Standard

for Information Technology – Cybersecurity – Secure Software Development Framework

INCITS 585-2025

Developed by

incits™

Where IT all begins

Approved American National Standard

ANSI

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

INCITS 585-2025

American National Standard
for Information Technology –

Cybersecurity –
Secure Software Development Framework

Secretariat

Information Technology Industry Council

Approved September 26, 2025

American National Standards Institute, Inc.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

American National Standard

Approval of an American National Standard requires review by ANSI that the requirements for due process, consensus, and other criteria for approval have been met by the standards developer.

Consensus is established when, in the judgment of the ANSI Board of Standards Review, substantial agreement has been reached by directly and materially affected interests. Substantial agreement means much more than a simple majority, but not necessarily unanimity. Consensus requires that all views and objections be considered, and that a concerted effort be made towards their resolution.

The use of American National Standards is completely voluntary; their existence does not in any respect preclude anyone, whether he has approved the standards or not, from manufacturing, marketing, purchasing, or using products, processes, or procedures not conforming to the standards.

The American National Standards Institute does not develop standards and will in no circumstances give an interpretation of any American National Standard. Moreover, no person shall have the right or authority to issue an interpretation of an American National Standard in the name of the American National Standards Institute. Requests for interpretations should be addressed to the ANSI-accredited standards developer whose name appears on the title page of this standard.

CAUTION NOTICE: This American National Standard may be revised or withdrawn at any time. The procedures of the American National Standards Institute require that action be taken periodically to reaffirm, revise, or withdraw this standard. Purchasers of American National Standards may receive current information on all standards by calling or writing the American National Standards Institute.

NOTE – The user's attention is called to the possibility that compliance with this standard may require use of an invention covered by patent rights.

By publication of this standard, no position is taken with respect to the validity of any such claim(s) or of any patent rights in connection therewith. If a patent holder has filed a statement of willingness to grant a license under these rights on reasonable and nondiscriminatory terms and conditions to applicants desiring to obtain such a license, then details may be obtained from the standards developer.

Published by

American National Standards Institute, Inc.
25 West 43rd Street, New York, NY 10036

Copyright © 2025 by Information Technology Industry Council (ITI)
All rights reserved.

No part of this publication may be reproduced in any
form, in an electronic retrieval system or otherwise,
without prior written permission of ITI, 700 K Street NW, Suite 600,
Washington, DC 20001.

Printed in the United States of America

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Contents

Page

Foreword v

Introduction xi

1 Scope 1

2 Normative references 1

3 Terms and definitions and abbreviations 1

4 Background 2

4.1 Approach 2

4.2 SSDF Use 3

4.3 The Secure Software Development Framework 3

5 Prepare the Organization 4

5.1 Define Security Requirements for Software Development 4

5.1.1 Identify and document security requirements for software development infrastructures and processes 4
5.1.2 Identify and document security requirements for developed software 5
5.1.3 Communicate requirements 5

5.2 Implement Roles and Responsibilities 5
5.2.1 Update role definitions 5
5.2.2 Provide training 5
5.2.3 Obtain and convey management commitment 5

5.3 Implement Supporting Toolchains 5

5.3.1 Specify tools 5
5.3.2 Secure tools and toolchains 5
5.3.3 Configure tools to generate artifacts 5

5.4 Define and Use Criteria for Software Security Checks 5
5.4.1 Define security check criteria 6
5.4.2 Gather and safeguard information for security checks 6

5.5 Implement and Maintain Secure Environments for Software Development 6
5.5.1 Separate and protect environments 6
5.5.2 Secure and harden endpoints 6

6 Protect the Software 6

6.1 Protect All Forms of Code from Unauthorized Access and Tampering 6
6.1.1 Securely store code 6

6.2 Provide a Mechanism for Verifying Software Release Integrity 6

6.2.1 Share software integrity verification information 6

6.3 Archive and Protect Each Software Release 6
6.3.1 Retain software releases 7
6.3.2 Track software provenance data 7

7 Produce Well-Secured Software 7

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Page

7.1 Design Software to Meet Security Requirements and Mitigate Security Risks ... 7
7.1.1 Model the security risk ... 7
7.1.2 Track design decisions ... 7
7.1.3 Use standardized security features and services ... 7

7.2 Review the Software Design to Verify Compliance with Security Requirements and Risk Information ... 7
7.2.1 Validate the software design ... 7

7.3 Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality ... 7
7.3.1 Maintain well-secured third-party software components ... 8
7.3.2 Create and maintain well-secured in-house software components ... 8
7.3.3 Verify security of third-party software components ... 8

7.4 Create Source Code by Adhering to Secure Coding Practices ... 8
7.4.1 Follow appropriate secure coding practices ... 8

7.5 Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security ... 8
7.5.1 Choose compiler, interpreter, and build tools ... 8
7.5.2 Configure compiler, interpreter, and build tools ... 8

7.6 Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements ... 8
7.6.1 Determine when to review or analyze code ... 8
7.6.2 Review and analyze code ... 9

7.7 Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements ... 9

7.7.1 Determine when and how to test code ... 9

7.7.2 Test code ... 9

7.8 Configure Software to Have Secure Settings by Default ... 9
7.8.1 Define a secure baseline ... 9
7.8.2 Implement and document the secure baseline ... 9

8 Respond to Vulnerabilities ... 9

8.1 Identify and Confirm Vulnerabilities on an Ongoing Basis ... 9

8.1.1 Investigate vulnerability reports ... 10
8.1.2 Evaluate software for vulnerabilities ... 10
8.1.3 Implement vulnerability disclosure and remediation ... 10

8.2 Assess, Prioritize, and Remediate Vulnerabilities ... 10
8.2.1 Analyze vulnerabilities ... 10
8.2.2 Implement risk responses ... 10

8.3 Analyze Vulnerabilities to Identify Their Root Causes ... 10
8.3.1 Determine root causes ... 10
8.3.2 Identify patterns with root causes ... 10
8.3.3 Eradicate similar vulnerabilities ... 10
8.3.4 Update the SDLC process ... 10

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Page

Bibliography 11

Annex A Implementation examples 12
A.1 Prepare the Organization 12
A.1.1 Define security requirements for software development 12
A.1.1.1 Identify and document security requirements for software development infrastructures and processes 12
A.1.1.2 Identify and document security requirements for developed software 12
A.1.1.3 Communicate requirements 13

A.1.2 Implement Roles and Responsibilities 13
A.1.2.1 Update role definitions 13
A.1.2.2 Provide training 13
A.1.2.3 Obtain and convey management commitment 14

A.1.3 Implement Supporting Toolchains 14
A.1.3.1 Specify tools 14
A.1.3.2 Secure tools and toolchains 14
A.1.3.3 Configure tools to generate artifacts 14

A.1.4 Define and Use Criteria for Software Security Checks 15
A.1.4.1 Define security check criteria 15
A.1.4.2 Gather and safeguard information for security checks 15

A.1.5 Implement and Maintain Secure Environments for Software Development 15
A.1.5.1 Separate and protect environments 16
A.1.5.2 Secure and harden endpoints 16

A.2 Protect the Software 16
A.2.1 Protect all forms of code from unauthorized access and tampering 16
A.2.1.1 Securely store code 16

A.2.2 Provide a Mechanism for Verifying Software Release Integrity 16
A.2.2.1 Share software integrity verification information 16

A.2.3 Archive and Protect Each Software Release 17
A.2.3.1 Retain software releases 17
A.2.3.2 Share software provenance data 17

A.3 Produce Well Secured Software 17

A3,1 Design Software to Meet Security Requirements and Mitigate Security Risks 17
A.3.1.1Model the security risk 17
A.3.1.2Track design decisions 17
A.3.1.3Use standardized security features and services 17

A.3.2Review the Software Design to Verify Compliance with Security Requirements and Risk Information 18
A.3.2.1Validate the software design 18

A.3.3Reuse Existing, Well Secured Software when Feasible Instead of Duplicating Functionality 18
A.3.3.1Maintain well secured third-party software components 18
A.3.3.2Create and maintain well secured in-house software components 18
A.3.3.3Verify security of third party software components 19

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Page

A.3.4 Create Source Code by Adhering to Secure Coding Practices 19
A.3.4.1 Follow appropriate secure coding practices 19

A.3.5 Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security 19
A.3.5.1 Choose compiler, interpreter and build tools 19
A.3.5.2 Configure compiler, interpreter and build tools 20

A.3.6 Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements 20
A.3.6.1 Determine when to review or analyze code 20
A.3.6.2 Review and analyze code 20

A.3.7 Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements 21
A.3.7.1 Determine when and how to test code 21
A.3.7.2 Test code 21

A.3.8 Configure Software to Have Secure Settings by Default 21
A.3.8.1 Define a secure baseline 21
A.3.8.2 Implement and document the secure baseline 21

A.4 Respond to Vulnerabilities 22

A.4.1 Identify and Confirm Vulnerabilities on an Ongoing Basis 22
A.4.1.1 Investigate vulnerability reports 22
A.4.1.2 Evaluate software for vulnerabilities 22
A.4.1.3 Implement vulnerability disclosure and remediation 22

A.4.2 Assess, Prioritize, and Remediate Vulnerabilities 22
A.4.2.1 Analyze vulnerabilities 22
A.4.2.2 Implement risk responses 22

A.4.3 Analyze Vulnerabilities to Identify Their Root Causes 23
A.4.3.1 Determine root causes 23
A.4.3.2 Identify patterns with root causes 23
A.4.3.3 Eradicate similar vulnerabilities 23
A.4.3.4 Update the SDLC process 23

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Foreword (This foreword is not part of American National Standard INCITS 585-2025.)

The Secure Software Development Framework describes a core set of high-level secure software development practices that can be integrated into any software development life cycle (SDLC) implementation. Following such practices should help software producers reduce the number of vulnerabilities in released software, reduce the potential impact of the exploitation of undetected or unaddressed vulnerabilities, and address the root causes of vulnerabilities to prevent future recurrences. Because the framework provides a common vocabulary for secure software development, software acquirers can also use it to foster communications with suppliers in acquisition processes and other management activities. This standard was developed by Technical Committee Cybersecurity and Privacy of Accredited Standards Committee INCITS during 2023-2024. The standards approval process started in 2023.

Requests for interpretation, suggestions for improvement and addenda, or defect reports are welcome. They should be sent to the INCITS Secretariat, National Committee for Information Technology Standards, Information Technology Industry Council, 700 K Street NW, Suite 600,, Washington, DC 20001.

This standard was processed and approved for submittal to ANSI by the InterNational Committee for Information Technology Standards (INCITS). Committee approval of the standard does not necessarily imply that all committee members voted for approval. At the time it approved this standard, INCITS had the following members:

Laura Lindsay, Chair
Anish Karmarkar, Vice-Chair
Jennifer Garner, Secretary

Organization Represented	*Name of Representative*
Adobe Systems, Inc.	Scott Foshee
	David Sankel (Alt.)
Amazon Web Services, Inc.	Anish Karmarkar
Apple	Helene Workman
	Hicham Lozi (Alt.)
	Michael Spencer (Alt.)
	Anna Weinberg (Alt.)
Department of Commerce - NIST	Sal Francomacaro
	Lisa Carnahan (Alt.)
	Jessica Dickson (Alt.)
	Barbara Goldstein (Alt.)
	Lauren Molina (Alt.)
	Anthony Quinn (Alt.)
	Adam Sedgewick (Alt.)
Farance, Inc.	Frank Farance
	Timothy Schoechle (Alt.)
	Sue Ellen Wright (Alt.)
Futurewei Technologies, Inc.	Wael Diab
	Wilbert Adams (Alt.)
	Timothy Jeffries (Alt.)
Google	Lenora Zimmerman
	John Britton (Alt.)
	Chandler Carruth (Alt.)
GS1GO	Steven Keddie
	Edward Merrill (Alt.)
	Dan Mullen (Alt.)
HP, Inc.	Muhammad Ali
	Paul Jeran (Alt.)
IBM Corporation	Arnaud Le Hors
	Alexander Tarpinian (Alt.)

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Organization Represented	*Name of Representative*
Microsoft Corporation	Laura Lindsay
	John Calhoon (Alt.)
	Scott Edwards (Alt.)
Oracle	Alex Li
	Michael Kavanaugh (Alt.)
	Luke Kowalski (Alt.)
	Jan-Eike Michels (Alt.)
	Heather Vancura (Alt.)
Qualcomm, Inc.	Michael Atlass
	Marta Karczewicz (Alt.)
Sandflow Consulting LLC	Pierre-Anthony Lemieux
U.S. Department of Defense	Jay Chung
	James Andrukaitis (Alt.)
	Jessica Fitzgerald-McKay (Alt.)

Emeritus

Donald Deutsch
Phil Wennblom

INCITS Technical Committee on Cybersecurity and Privacy, which developed and reviewed this standard, had the following members:

Eric Hibbard, Chair
John Britton, Vice-Chair
Thomas Rivera, Secretary
Nadya Bartol, International Representative

Organization Represented	*Name of Representative*
A-LIGN Compliance and Security, Inc.	Patrick Sullivan
Amazon Web Services, Inc.	Anish Karmarkar
	Sergul Aydore (Alt.)
	Matthew Campagna (Alt.)
	Greg Cannon (Alt.)
	Ryan DeSouza (Alt.)
	Richard Gardner (Alt.)
	Shai Halevi (Alt.)
	Judd Hesselroth (Alt.)
	Elaine Newton (Alt.)
	Kelly Nieto (Alt.)
	Brian Olion (Alt.)
	Abby Palia (Alt.)
	Pinar Yilmaz (Alt.)
Apple	Samuel Dillon
	Lanpeng Chen (Alt.)
	Nick Kucera (Alt.)
	Huarui Liang (Alt.)
	Jill Schmidt (Alt.)
	Yannick Sierra (Alt.)
	Chris Wood (Alt.)
Atsec Information Security Corporation	Yi Mao
	Andreas Fabis (Alt.)
	Marcos Portnoi (Alt.)
BCG	Nadya Bartol
	Alex Asen (Alt.)
	Sean Mitchell (Alt.)
	Evan O'Reilly (Alt.)
Cisco Systems, Inc.	Peter Schulz
	Lisa Bobbitt (Alt.)
	Ken Dill (Alt.)
	Jonathan Fox (Alt.)
	Jonathan Kessler (Alt.)
	Ekta Mittal (Alt.)
Consumer Technology Association (CTA) (Liaison)	Mike Bergman
	Veronica Lancaster

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Organization Represented	*Name of Representative*
Dell, Inc.	Said Tabet
	Erin Bournival (Alt.)
	Edward Connolly (Alt.)
	Judith Furlong (Alt.)
	Joe Laugle (Alt.)
	Steven Schmalz (Alt.)
	Tania Ward (Alt.)
Deloitte & Touche LLP	Christopher Williams
	Casper Stap (Alt.)
Department of Commerce - NIST	Sal Francomacaro
	Christine Abruzzi (Alt.)
	Meghan Anderso (Alt.)
	Jon Boyens (Alt.)
	Alexander Calis (Alt.)
	Ramaswamy Chandramouli (Alt.)
	Lily Chen (Alt.)
	Frederic de Vaulx (Alt.)
	Jessica Dickson (Alt.)
	Jesse Dunietz (Alt,)
	David Ferraiolo (Alt.)
	Hildegard Ferraiolo (Alt.)
	Ryan Galluzzo (Alt.)
	Dylan Gilbert (Alt.)
	Nedim Goren (Alt.)
	Patrick Grother (Alt.)
	Michaela Iorga (Alt.)
	Connie LaSalle (Alt.)
	Suzanne Lightman (Alt.)
	Ben Livelsberger (Alt.)
	Amy Mahn (Alt.)
	Katerina Megas (Alt.)
	Ketan Mehta (Alt.)
	Nicky Mouha (Alt.)
	Michael Pease (Alt.)
	Victoria Pillitteri (Alt.)
	Andrew Regenscheid (Alt.)
	Ron Ross (Alt.)
	Matthew Smith (Alt.)
	Murugiah Souppaya (Alt.)
	Eduardo Takamura (Alt.)
	David Temoshok (Alt.)
	Laurin Weissinger (Alt.)
Exponent, Inc.	Brad McGoran
Farance, Inc.	Brandy Smith
Futurewei Technologies, Inc.	Barry Leiba
	Wael Diab (Alt.)
	John Strassner (Alt.)
	Lei Wang (Alt.)
Google	Lenora Zimmerman
	John Britton (Alt.)
	Dave Kleidermacher (Alt.)
	Esther Lim (Alt.)
	Tania Melo (Alt.)
	Catherine Nelson (Alt.)
	Nataliya Stanetsky (Alt.)
	Rachelle Summers (Alt.)
	Brian Wood (Alt.)
	QiQi Xue (Alt.)
Hitachi Vantara	Brian Williams
HP, Inc	Aaron Weller
	Muhammad Ali (Alt.)
	Paul Anderson (Alt.)
IBM Corporation	William Santiago

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Organization Represented	*Name of Representative*
Intel Corporation	Grace Wei
	Thomas Bowen (Alt.)
	Rosario Cammarota (Alt.)
	Josh Dembling (Alt.)
	Huijing Gong (Alt.)
	Duhyeong Kim (Alt.)
	Gerrit Kruitbosch (Alt.)
	Jun Takei (Alt.)
	Anahit Tarkhanyan (Alt.)
	Wen Wang (Alt.)
International Council on System Engineering (INCOSE)	Richard Martin
	Robert Brouwers (Alt.)
	Sean McCoy (Alt.)
	Thom Schoeffling (Alt.)
	Edward Yakabovicz (Alt.)
	Kenneth Zemrowski (Alt.)
Kaiser Permanente	Jamie Fergusen
	Thanh-Thien Nguyen (Alt.)
	Kimberly Pagani (Alt.)
Kantara Initiative (Liaison)	Kav Chopard
Legrand US	Dale Roberts
	Grace Tang (Alt.)
Lexmark International	Will Anderson
	Bryan Willett (Alt.)
	Don Wright (Alt.)
Lockheed Martin Corporation	Chris Wood
	Dawn Beyer (Alt.)
	Richard Forselius (Alt.)
	Marilyn Pineda (Alt.)
Lorica Systems, Inc.	Luis Ruiz-Lopez
Meta Platforms, Inc.	Rafael Misoczki
Microsoft Corporation	Laura Lindsay
	Juliana Cafik (Alt.)
	Pamela Dingle (Alt.)
	Karen Easterbrook (Alt.)
	Jill Friar (Alt.)
	Mikyung Gioanni (Alt.)
	Mike Grimm (Alt.)
	Pieter Kasselman (Alt.)
	Lindsey LaGesse (Alt.)
	Kim Laine (Alt.)
	Patrick Longa (Alt.)
	Kimberly Lucy (Alt.)
	Rob Spiger (Alt.)
	Mark Svancarek (Alt.)
	Steve Venema (Alt.)
	Carla Weitkamp (Alt.)
Moss Adams	Bryan Schader
National Electrical Manufacturers Association (NEMA)	Emily Hoefer
	Hae Choe (Alt.)
NES Consulting	Musab Gunes
NetApp, Inc.	Sridhar Balasubramanian
nuTonomy, Inc.	Charles Wilson
Oracle	John Routh
	Alex Li (Alt.)
	Tyrone Stodart (Alt.)
	Betina Tagle (Alt.)
Pacific Northwest University - Cyber Education	Anthony Nadalin

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Organization Represented	*Name of Representative*
PrivSec Consulting LLC	Ian Hibbard
	Eva Kuiper (Alt.)
	Andrew Nielsen (Alt.)
	Thomas Rivera (Alt.)
	Fiona Stewart (Alt.)
	Tomone Tanaka (Alt.)
	Steven Teppler (Alt.)
	Lucy Thomson (Alt.)
	Richard Wilsher (Alt.)
QRC America LLC	Christy Raedeke
Ricoh Corporation	Brian Volkoff
RTX	Randall Brooks
	Kelly FitzGerald (Alt.)
	Noel Johnson (Alt.)
	Fred Jones (Alt.)
	Dan Williams (Alt.)
Samsung Semiconductor, Inc. (SSI)	Eric Hibbard
	Khaled Ahmed (Alt.)
	Ranjit Mallela (Alt.)
	William Martin (Alt.)
Schellman Compliance LLC	Ryan Mackie
	Kristin Hric (Alt.)
	Jason Lam (Alt.)
	Danny Manimbo (Alt.)
Securisea, Inc.	Josh Daymont
Software Engineering Institute/CERT, a division of Carnegie Mellon University	Sandi Noonan
	Ryan Karl (Alt.)
	Sheila Rosenthal (Alt.)
	Robin Ruefle (Alt.)
	Laurie Tyzenhaus (Alt.)
	Nathan Vanhoudnos (Alt.)
	Charles Yarbrough (Alt.)
Sylint Group	Serge Jorgensen
	Charles Shugg (Alt.)
The Aerospace Corporation	Meredith Martinez
The Open Group (Liaison)	Joseph Bergmann
Uber Technologies, Inc.	Bill Harmon
	Samantha Hsu (Alt.)
Underwriters Laboratories, Inc.	Denice Durrant
United States Dept. of Defense	Jay Yoo
	Rampersaud Ramnarayan (Alt.)
United States Dept. of Defense - NSA	Nick Gajcowski
	Inna Baron (Alt.)
	David Bickell (Alt.)
	Laura Corcoran (Alt.)
	Rich Davis (Alt.)
	Jessica Fitzgerald-McKay (Alt.)
	Patrick Gallo (Alt.)
	Michael Hernandez (Alt.)
	Dylan Kehs (Alt.)
	Jenave Minter (Alt.)
	Greg Ott (Alt.)
	Stephanie Polczynski (Liaison)
	Alexander Rabinowitz (Alt.)
	Kayla Robb (Alt.)
	Michael Rosa (Alt.)
	Jade Stewart (Alt.)

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Organization Represented	*Name of Representative*
United States Dept. of Homeland Security	Juan Gonzalez Anil John (Alt.) Yonas Nebiveloul-Kifle (Alt.) Arun Vemury (Alt.)
VISA	Nirmal Baid Kimberly Bella (Alt.) Kristina Breen (Alt.) Adam Clark (Alt.) Marie Jordan (Alt.) Kim Wagner (Alt.)

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Introduction

A software development life cycle (SDLC) is a formal or informal methodology for designing, creating, and maintaining software (including code built into hardware). There are many models for SDLCs, including waterfall, spiral, agile, and agile combined with software development and IT operations (DevOps) practices. Regardless of which SDLC model is used, secure software development practices should be integrated throughout it for three reasons: to reduce the number of vulnerabilities in released software, to reduce the potential impact of the exploitation of undetected or unaddressed vulnerabilities, and to address the root causes of vulnerabilities to prevent recurrences. Vulnerabilities include not just bugs caused by coding flaws, but also weaknesses caused by security configuration settings, incorrect trust assumptions, and outdated risk analysis.

Most aspects of security can be addressed multiple times within an SDLC, but in general, the earlier in the SDLC that security is addressed, the less effort and cost is ultimately required to achieve the same level of security. This principle, known as shifting left, is critically important regardless of the SDLC model. Shifting left minimizes any technical debt that would require remediating early security flaws late in development or after the software is in production. Shifting left can also result in software with stronger security and resiliency.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

American National Standard
for Information Technology –

Cybersecurity –
Secure Software Development Framework

1 Scope

This document provides a framework for secure software development practices.

2 Normative references

There are no normative references in this document.

3 Terms and definitions and abbreviations

ISO and IEC maintain terminology databases for use in standardization at the following addresses:

— ISO Online browsing platform: available at https://www.iso.org/obp
— IEC Electropedia: available at https://www.electropedia.org/

3.1
artifact
a piece of evidence

Note 1 to entry: Artifacts provide records of secure software development practices.

3.2
environment
context determining the setting and circumstances of all influences upon a system

[SOURCE: ISO/IEC/IEEE 42010:2011]

3.3
provenance
information on the place and time of origin, derivation or generation of a resource or a record or proof of authenticity or of past ownership

[SOURCE: ISO/IEC 11179-7:2019]

3.4
secure software development framework
high-level practices based on established standards, guidance, and secure software development practice documents.

3.5
software development life cycle
a formal or informal methodology for designing, creating, and maintaining software (including code built into hardware).

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

3.6
stakeholder
person or organization that can affect, be affected by, or perceive itself to be affected by a decision or activity

[SOURCE: ISO/IEC 27000:2018]

Abbreviations

SDLC software development life cycle

SSDF Secure Software Development Framework

4 Background

This document describes a set of high-level secure software development practices based on established standards, guidance, and other documents. These practices, collectively called the Secure Software Development Framework (SSDF), are intended to help the target audiences achieve secure software development objectives. Many of the practices directly involve the software itself, while others indirectly involve it (e.g., securing the development environment).

This document provides a common language to describe fundamental secure software development practices. Expertise in secure software development is not required to understand the practices. The common language helps facilitate communications about secure software practices among both internal and external organizational stakeholders, such as:

- Business owners, software developers, project managers and leads, cybersecurity professionals, and operations and platform engineers within an organization who need to clearly communicate with each other about secure software development
- Software acquirers that want to define required or desired characteristics for software in their acquisition processes in order to have higher-quality software (particularly with fewer significant security vulnerabilities)
- Software producers (e.g., commercial-off-the-shelf [COTS] product vendors, government-off-the-shelf [GOTS] software developers, software developers working within or on behalf of software acquirer organizations) that want to integrate secure software development practices throughout their SDLCs, express their secure software practices to their customers, or define requirements for their suppliers

4.1 Approach

This document identifies secure software development practices but does not prescribe how to implement them. The focus is on the outcomes of the practices to be implemented rather than on the tools, techniques, and mechanisms used to do so. Advantages of specifying the practices at a high level include the following:

- Can be used by organizations in any sector or community, regardless of size or cybersecurity sophistication
- Can be applied to software developed to support information technology (IT), artificial intelligence (AI), industrial control systems (ICS), cyber-physical systems (CPS), or the Internet of Things (IoT)

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

- Can be integrated into any existing software development workflow and automated toolchain; should not negatively affect organizations that already have robust secure software development practices in place
- Makes the practices broadly applicable, not specific to particular technologies, platforms, programming languages, SDLC models, development environments, operating environments, tools, etc.
- Can help an organization document its secure software development practices today and define its future target practices as part of its continuous improvement process
- Can assist an organization currently using a classic software development model in transitioning its secure software development practices for use with a modern software development model (e.g., agile, DevOps)
- Can assist organizations that are procuring and using software to understand secure software development practices employed by their suppliers

4.2 SSDF Use

This document's practices are not based on the assumption that all organizations have the same security objectives and priorities. Rather, the recommendations reflect that each software producer may have unique security requirements, and each software acquirer may have unique security needs and requirements. While the aim is for each software producer to follow all applicable practices, the expectation is that the degree to which each practice is implemented and the formality of the implementation will vary based on the producer's security requirements. The practices provide flexibility for implementers, but they are also clear to avoid leaving too much open to interpretation.

Although most of these practices are relevant to any software development effort, some are not. For example, if developing a particular piece of software does not involve using a compiler, there would be no need to follow a practice on configuring the compiler to improve executable security. Some practices are foundational, while others are more advanced and depend on certain foundational practices already being in place. Also, practices are not all equally important for all cases.

Factors such as risk, cost, feasibility, and applicability should be considered when deciding which practices to use and how much time and resources to devote to each practice. Automatability is also an important factor to consider, especially for implementing practices at scale. The practices, tasks, and implementation examples (implementation examples are in Annex A) represent a starting point to consider; they are meant to be changed and customized, and they are not prioritized. Any stated frequency for performing practices is notional. The intention of the SSDF is not to create a checklist to follow, but to provide a basis for planning and implementing a risk-based approach to adopting secure software development practices.

The responsibility for implementing the practices may be distributed among different organizations based on the delivery of the software and services (e.g., infrastructure as a service, software as a service, platform as a service, container as a service, serverless). In these situations, it likely follows a shared responsibility model involving the platform/service providers and the tenant organization that is consuming those platforms/services. The tenant organization should establish an agreement with the providers that specifies which party is responsible for each practice and task and how each provider will attest to their conformance with the agreement.

4.3 The Secure Software Development Framework

This document defines the Secure Software Development Framework (SSDF) with fundamental, sound, and secure recommended practices based on established secure

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

software development practice documents. The practices are organized into four *practice groups*:

Prepare the Organization: Organizations should ensure that their people, processes, and technology are prepared to perform secure software development at the organization level. Many organizations will find some Prepare the Organization practices to also be applicable to subsets of their software development, like individual development groups or projects. Prepare the Organization provides a foundation for the other practice groups, as **Fig. 1** depicts.

Protect the Software: Organizations should protect all components of their software from tampering and unauthorized access.

Produce Well-Secured Software: Organizations should produce well-secured software with minimal security vulnerabilities in its releases.

Respond to Vulnerabilities: Organizations should identify residual vulnerabilities in their software releases and respond appropriately to address those vulnerabilities and prevent similar ones from occurring in the future.

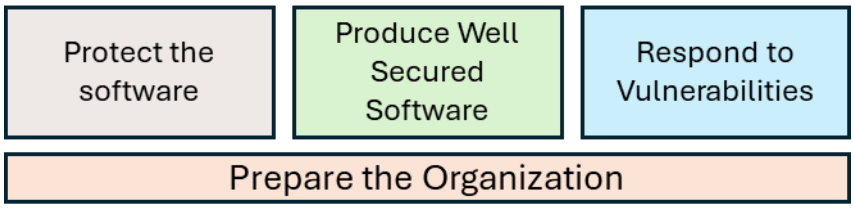

Figure 1. SSDF Practice Groups

Each practice group contains multiple *practices*. Each practice definition includes the name of the practice, followed by a brief explanation of what the practice is and why it is beneficial. Each practice also defines one or more *tasks*, which are actions that may be needed to perform the practice. A task definition includes an explanation of the task.

The rest of this document defines the practices. Note that the order of the practices and tasks is not intended to imply the sequence of implementation or the relative importance of any practice or task.

5 Prepare the Organization

Organizations should ensure that their people, processes, and technology are prepared to perform secure software development at the organization level. Many organizations will find some of these practices to also be applicable to subsets of their software development, like individual development groups or projects.

5.1 Define Security Requirements for Software Development

Ensure that security requirements for software development are known at all times so that they can be taken into account throughout the SDLC and requirements information can be collected once and shared to minimize the duplication of effort. This includes requirements from internal sources (e.g., the organization's policies, business objectives, and risk management strategy) and external sources (e.g., applicable laws and regulations).

5.1.1 Identify and document security requirements for software development infrastructures and processes

Identify and document all security requirements for the organization's software development infrastructures and processes, and maintain the requirements over time.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

5.1.2 Identify and document security requirements for developed software

Identify and document all security requirements for organization-developed software to meet, and maintain the requirements over time.

5.1.3 Communicate requirements

Communicate requirements to all third parties who will provide commercial software components to the organization for reuse by the organization's own software.

5.2 Implement Roles and Responsibilities

Ensure that everyone inside and outside of the organization involved in the SDLC is prepared to perform their SDLC-related roles and responsibilities throughout the SDLC.

5.2.1 Update role definitions

Create new roles and alter responsibilities for existing roles as needed to encompass all parts of the SDLC. Periodically review and maintain the defined roles and responsibilities, updating them as needed.

5.2.2 Provide training

Provide role-based training for all personnel with responsibilities that contribute to secure development. Periodically review personnel proficiency and role-based training, and update the training as needed.

5.2.3 Obtain and convey management commitment

Obtain upper management or authorizing official commitment to secure development and convey that commitment to those with development related roles and responsibilities.

5.3 Implement Supporting Toolchains

Use automation to reduce human effort and improve the accuracy, reproducibility, usability, and comprehensiveness of security practices throughout the SDLC, as well as provide a way to document and demonstrate the use of these practices. Toolchains and tools may be used at different levels of the organization, such as organization-wide or project-specific, and may address a particular part of the SDLC, like a build pipeline.

5.3.1 Specify tools

Specify which tools or tool types must or should be included in each toolchain to mitigate identified risks, as well as how the toolchain components are to be integrated with each other.

5.3.2 Secure tools and toolchains

Follow recommended security practices to deploy, operate, and maintain tools and toolchains.

5.3.3 Configure tools to generate artifacts

Configure tools to generate artifacts representing evidence of secure software development practices as defined by the organization.

5.4 Define and Use Criteria for Software Security Checks

Help ensure that the software resulting from the SDLC meets the organization's expectations by defining and using criteria for checking the software's security during development.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

5.4.1 Define security check criteria

Define criteria for software security checks and track throughout the SDLC.

5.4.2 Gather and safeguard information for security checks

Implement processes, mechanisms, and controls to gather and safeguard the necessary information in support of the criteria.

5.5 Implement and Maintain Secure Environments for Software Development

Ensure that all components of the environments for software development are strongly protected from internal and external threats to prevent compromises of the environments or the software being developed or maintained within them. Examples of environments for software development include development, build, test, and distribution environments.

5.5.1 Separate and protect environments

Separate and protect each environment involved in software development.

5.5.2 Secure and harden endpoints

Secure and harden development endpoints (i.e., endpoints for software designers, developers, testers, builders, etc.) to perform development-related tasks using a risk-based approach.

6 Protect the Software

Organizations should protect all components of their software from tampering and unauthorized access.

6.1 Protect All Forms of Code from Unauthorized Access and Tampering

Help prevent unauthorized changes to code, both inadvertent and intentional, which could circumvent or negate the intended security characteristics of the software. Code not intended to be publicly accessible is accessible only for authorized access to help prevent theft of the software and potentially make it more difficult or time-consuming for attackers to find vulnerabilities in the software.

6.1.1 Securely store code

Store all forms of code – including source code, executable code, and configuration-as-code – based on the principle of least privilege so that only authorized personnel, tools, services, etc. have access.

6.2 Provide a Mechanism for Verifying Software Release Integrity

Help software acquirers ensure that the software they acquire is legitimate and has not been tampered with.

6.2.1 Share software integrity verification information

Make software integrity verification information available to software acquirers (e.g., cryptographic checksums or digital signatures captured in a machine-readable data format that can be used with a software bill of materials [SBOM] or a provenance tracking system).

6.3 Archive and Protect Each Software Release

Preserve software releases in order to help identify, analyze, and eliminate vulnerabilities discovered in the software after release.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

6.3.1 Retain software releases

Securely archive the necessary files and supporting data (e.g., integrity verification information, provenance data) to be retained for each software release.

6.3.2 Track software provenance data

Collect, safeguard, maintain, and track provenance data for all components of each software release as artifacts for traceability and sharing when appropriate (e.g., in a software bill of materials [SBOM] or provenance tracking system).

7 Produce Well-Secured Software

Organizations should produce well-secured software with minimal security vulnerabilities in its releases.

7.1 Design Software to Meet Security Requirements and Mitigate Security Risks

Identify and evaluate the security requirements for the software; determine what security risks the software is likely to face during operation and how the software's design and architecture should mitigate those risks; and document any cases where risk-based analysis indicates that security requirements should be relaxed or waived. Addressing security requirements and risks during software design (secure by design) is key for improving software security and also helps improve development efficiency.

7.1.1 Model the security risk

Use forms of risk modeling – such as threat modeling, attack modeling, or attack surface mapping – to help assess the security risk for the software.

7.1.2 Track design decisions

Track and maintain the software's security requirements, risks, and design decisions.

7.1.3 Use standardized security features and services

Where appropriate, build in support for using standardized security features and services (e.g., enabling software to integrate with existing log management, identity management, access control, and vulnerability management systems) instead of creating proprietary implementations of security features and services.

7.2 Review the Software Design to Verify Compliance with Security Requirements and Risk Information

Help ensure that the software will meet the security requirements and satisfactorily address the identified risk information.

7.2.1 Validate the software design

Have 1) a qualified person (or people) who were not involved with the design and/or 2) automated processes instantiated in the toolchain review the software design to confirm and enforce that it meets all of the security requirements and satisfactorily addresses the identified risk information.

7.3 Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality

Lower the costs of software development, expedite software development, and decrease the likelihood of introducing additional security vulnerabilities into the software by reusing

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

software modules and services that have already had their security posture verified. This is particularly important for software that implements security functionality, such as cryptographic modules and protocols.

7.3.1 Maintain well-secured third-party software components

Acquire and maintain well-secured software components (e.g., software libraries, modules, middleware, frameworks) from commercial, opensource, and other third-party developers for use by the organization's software.

7.3.2 Create and maintain well-secured in-house software components

Create and maintain well-secured software components in-house following SDLC processes to meet common internal software development needs that cannot be better met by third-party software components.

7.3.3 Verify security of third-party software components

Verify that acquired commercial, open-source, and all other third-party software components comply with the requirements, as defined by the organization, throughout their life cycles.

7.4 Create Source Code by Adhering to Secure Coding Practices

Decrease the number of security vulnerabilities in the software and reduce costs by minimizing vulnerabilities introduced during source code creation.

7.4.1 Follow appropriate secure coding practices

Follow all secure coding practices that are appropriate to the development languages and environment to meet the organization's requirements.

7.5 Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security

Decrease the number of security vulnerabilities in the software and reduce costs by eliminating vulnerabilities before testing occurs.

7.5.1 Choose compiler, interpreter, and build tools

Use compiler, interpreter, and build tools that offer features to improve executable security.

7.5.2 Configure compiler, interpreter, and build tools

Determine which compiler, interpreter, and build tool features should be used and how each should be configured, then implement and use the approved configurations.

7.6 Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements

Help identify vulnerabilities so that they can be corrected before the software is released to prevent exploitation. Using automated methods lowers the effort and resources needed to detect vulnerabilities. Human-readable code includes source code, scripts, and any other form of code that an organization deems human readable.

7.6.1 Determine when to review or analyze code

Determine whether code review (a person looks directly at the code to find issues) and/or code analysis (tools are used to find issues in code, either in a fully automated way or in conjunction with a person) should be used, as defined by the organization.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

7.6.2 Review and analyze code

Perform the code review and/or code analysis based on the organization's secure coding standards, and record and triage all discovered issues and recommended remediations in the development team's workflow or issue tracking system.

7.7 Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements

Help identify vulnerabilities so that they can be corrected before the software is released in order to prevent exploitation. Using automated methods lowers the effort and resources needed to detect vulnerabilities and improves traceability and repeatability. Executable code includes binaries, directly executed bytecode and source code, and any other form of code that an organization deems executable.

7.7.1 Determine when and how to test code

Determine whether executable code testing should be performed to find vulnerabilities not identified by previous reviews, analysis, or testing and, if so, which types of testing should be used.

7.7.2 Test code

Scope the testing, design the tests, perform the testing, and document the results, including recording and triaging all discovered issues and recommended remediations in the development team's workflow or issue tracking system.

7.8 Configure Software to Have Secure Settings by Default

Help improve the security of the software at the time of installation to reduce the likelihood of the software being deployed with weak security settings, putting it at greater risk of compromise.

7.8.1 Define a secure baseline

Define a secure baseline by determining how to configure each setting that influences security or a security-related setting so that the default settings are secure and do not weaken the security functions provided by the platform, network infrastructure, or services.

7.8.2 Implement and document the secure baseline

Implement the default settings (or groups of default settings, if applicable), and document each setting for software administrators

8 Respond to Vulnerabilities

Organizations should identify residual vulnerabilities in their software releases and respond appropriately to address those vulnerabilities and prevent similar ones from occurring in the future.

8.1 Identify and Confirm Vulnerabilities on an Ongoing Basis

Help ensure that vulnerabilities are identified more quickly so that they can be remediated more quickly in accordance with risk, reducing the window of opportunity for attackers.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

8.1.1 Investigate vulnerability reports

Gather information from software acquirers, users, and public sources on potential vulnerabilities in the software and third-party components that the software uses, and investigate all credible reports.

8.1.2 Evaluate software for vulnerabilities

Review, analyze, and/or test the software's code to identify or confirm the presence of previously undetected vulnerabilities.

8.1.3 Implement vulnerability disclosure and remediation

Have a policy that addresses vulnerability disclosure and remediation, and implement the roles, responsibilities, and processes needed to support that policy. Prepare software updates and implement a vulnerability disclosure process (ISO/IEC 29147:2018) which is appropriate for the products, handling vulnerability reports and incidents. Have a team and process in place to handle the responses to vulnerability reports and incidents. (ISO/IEC 30111:2019)

8.2 Assess, Prioritize, and Remediate Vulnerabilities

Help ensure that vulnerabilities are remediated in accordance with risk to reduce the window of opportunity for attackers.

8.2.1 Analyze vulnerabilities

Analyze each vulnerability to gather sufficient information about risk to plan its remediation or other risk response.

8.2.2 Implement risk responses

Plan and implement risk responses for vulnerabilities.

8.3 Analyze Vulnerabilities to Identify Their Root Causes

Help reduce the frequency of vulnerabilities in the future.

8.3.1 Determine root causes

Analyze identified vulnerabilities to determine their root causes.

8.3.2 Identify patterns with root causes

Analyze the root causes over time to identify patterns, such as a particular secure coding practice not being followed consistently.

8.3.3 Eradicate similar vulnerabilities

Review the software for similar vulnerabilities to eradicate a class of vulnerabilities, and proactively fix them rather than waiting for external reports.

8.3.4 Update the SDLC process

Review the SDLC process and update it if appropriate to prevent (or reduce the likelihood of) the root cause recurring in updates to the software or in new software that is created.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Bibliography

[1] NIST SP 800-218 Secure Software Development Framework (SSDF)

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Annex A

Implementation Examples

The implementation examples provide notional examples of concise, action-oriented steps to help achieve SSDF practices and tasks. Verbs used to express examples include define, analyze, review, implement, document, measure, verify, automate, enforce, determine, detect, and use. The examples are not a comprehensive list of all actions that an organization could take, nor do they represent a baseline of required actions to address cybersecurity risks. Some examples may not apply to certain organizations and situations. No examples or combinations of examples are required.

A.1 Prepare the Organization

A.1.1 Define Security Requirements for Software Development

A.1.1.1 Identify and document security requirements for software development infrastructures and processes

Define policies for securing software development infrastructures and their components, including development endpoints, throughout the SDLC and maintaining that security.

Define policies for securing software development processes throughout the SDLC and maintaining that security, including for open-source and other third-party software components utilized by software being developed.

Review and update security requirements at least annually, or sooner if there are new requirements from internal or external sources, or a major security incident targeting software development infrastructure has occurred.

Educate affected individuals on impending changes to requirements.

A.1.1.2 Identify and document security requirements for developed software

Define policies that specify risk-based software architecture and design requirements, such as making code modular to facilitate code reuse and updates; isolating security components from other components during execution; avoiding undocumented commands and settings; and providing features that will aid software acquirers with the secure deployment, operation, and maintenance of the software.

Define policies that specify the security requirements for the organization's software and verify compliance at key points in the SDLC (e.g., classes of software flaws verified by gates, responses to vulnerabilities discovered in released software).

Analyze the risk of applicable technology stacks (e.g., languages, environments, deployment models), and recommend or require the use of stacks that will reduce risk compared to others.

Define policies that specify what needs to be archived for each software release (e.g., code, package files, third-party libraries, documentation, data inventory) and how long it needs to be retained based on the SDLC model, software end-of-life, and other factors.

Ensure that policies cover the entire software life cycle, including notifying users of the impending end of software support and the date of software end-of-life.

Review all security requirements at least annually, or sooner if there are new requirements from internal or external sources, a major vulnerability is discovered in released software, or a major security incident targeting organization-developed software has occurred.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Establish and follow processes for handling requirement exception requests, including periodic reviews of all approved exceptions.

A.1.1.3 Communicate requirements

Define a core set of security requirements for software components, and include it in acquisition documents, software contracts, and other agreements with third parties.

Define security-related criteria for selecting software; the criteria can include the third party's vulnerability disclosure program and product security incident response capabilities or the third party's adherence to organization defined practices.

Require third parties to attest that their software complies with the organization's security requirements.

Require third parties to track, and when appropriate, share provenance data and integrity verification mechanisms for all components of their software.

Establish and follow processes to address risk when there are security requirements that third-party software components to be acquired do not meet; this should include periodic reviews of all approved exceptions to requirements.

A.1.2 Implement Roles and Responsibilities

A.1.2.1 Update role definitions

Define SDLC-related roles and responsibilities for all members of the software development team.

Integrate the security roles into the software development team.

Define roles and responsibilities for cybersecurity staff, security champions, project managers and leads, senior management, software developers, software testers, software assurance leads and staff, product owners, operations and platform engineers, and others involved in the SDLC.

Conduct an annual review of all roles and responsibilities.

Educate affected individuals on impending changes to roles and responsibilities, and confirm that the individuals understand the changes and agree to follow them.

Implement and use tools and processes to promote communication and engagement among individuals with SDLC-related roles and responsibilities, such as creating messaging channels for team discussions.

Designate a group of individuals or a team as the code owner for each project.

A.1.2.2 Provide training

Document the desired outcomes of training for each role.

Define the type of training or curriculum required to achieve the desired outcome for each role.

Create a training plan for each role.

Acquire or create training for each role; acquired training may need to be customized for the organization.

Measure outcome performance to identify areas where changes to training may be beneficial.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.1.2.3 Obtain and convey management commitment

Appoint a single leader or leadership team to be responsible for the entire secure software development process, including being accountable for releasing software to production and delegating responsibilities as appropriate.

Increase authorizing officials' awareness of the risks of developing software without integrating security throughout the development life cycle and the risk mitigation provided by secure development practices.

Assist upper management in incorporating secure development support into their communications with personnel with development-related roles and responsibilities.

Educate all personnel with development-related roles and responsibilities on upper management's commitment to secure development and the importance of secure development to the organization.

A.1.3 Implement Supporting Toolchains

A.1.3.1 Specify tools

Define categories of toolchains, and specify the mandatory tools or tool types to be used for each category.

Identify security tools to integrate into the developer toolchain.

Define what information is to be passed between tools and what data formats are to be used.

Evaluate tools' signing capabilities to create immutable records/logs for auditability within the toolchain.

Use automated technology for toolchain management and orchestration.

A.1.3.2 Secure tools and toolchains

Evaluate, select, and acquire tools, and assess the security of each tool. Integrate tools with other tools and existing software development processes and workflows.

Use code-based configuration for toolchains (e.g., pipelines-as-code, toolchains-as-code).

Implement the technologies and processes needed for reproducible builds.

Update, upgrade, or replace tools as needed to address tool vulnerabilities or add new tool capabilities.

Continuously monitor tools and tool logs for potential operational and security issues, including policy violations and anomalous behavior.

Regularly verify the integrity and check the provenance of each tool to identify potential problems.

A.1.3.3 Configure tools to generate artifacts

Use existing tooling (e.g., workflow tracking, issue tracking, value stream mapping) to create an audit trail of the secure development-related actions that are performed for continuous improvement purposes.

Determine how often the collected information should be audited, and implement the necessary processes.

Establish and enforce security and retention policies for artifact data.

Assign responsibility for creating any needed artifacts that tools cannot generate.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.1.4 Define and Use Criteria for Software Security Checks

A.1.4.1 Define security check criteria

Ensure that the criteria adequately indicate how effectively security risk is being managed.

Define key performance indicators (KPIs), key risk indicators (KRIs), vulnerability severity scores, and other measures for software security.

Add software security criteria to existing checks (e.g., the Definition of Done in agile SDLC methodologies).

Review the artifacts generated as part of the software development workflow system to determine if they meet the criteria.

Record security check approvals, rejections, and exception requests as part of the workflow and tracking system.

Analyze collected data in the context of the security successes and failures of each development project, and use the results to improve the SDLC.

A.1.4.2 Gather and safeguard information for security checks

Use the toolchain to automatically gather information that informs security decision-making.

Deploy additional tools if needed to support the generation and collection of information supporting the criteria.

Automate decision-making processes utilizing the criteria, and periodically review these processes.

Only allow authorized personnel to access the gathered information, and prevent any alteration or deletion of the information.

A.1.5 Implement and Maintain Secure Environments for Software Development

A.1.5.1 Separate and protect environments

Use multi-factor, risk-based authentication and conditional access for each environment.

Use network segmentation and access controls to separate the environments from each other and from production environments, and to separate components from each other within each non-production environment, in order to reduce attack surfaces and attackers' lateral movement and privilege/access escalation.

Enforce authentication and tightly restrict connections entering and exiting each software development environment, including minimizing access to the internet to only what is necessary.

Minimize direct human access to toolchain systems, such as build services. Continuously monitor and audit all access attempts and all use of privileged access.

Minimize the use of production-environment software and services from non-production environments.

Regularly log, monitor, and audit trust relationships for authorization and access between the environments and between the components within each environment.

Continuously log and monitor operations and alerts across all components of the development environment to detect, respond, and recover from attempted and actual cyber incidents.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Configure security controls and other tools involved in separating and protecting the environments to generate artifacts for their activities.

Continuously monitor all software deployed in each environment for new vulnerabilities, and respond to vulnerabilities appropriately following a risk based approach.

Configure and implement measures to secure the environments' hosting infrastructures following zero trust principles.

A.1.5.2 Secure and harden endpoints

Configure each development endpoint based on approved hardening guides, checklists, etc.

Configure each development endpoint and the development resources to provide the least functionality needed by users and services and to enforce the principle of least privilege.

Continuously monitor the security posture of all development endpoints, including monitoring and auditing all use of privileged access.

Configure security controls and other tools involved in securing and hardening development endpoints to generate artifacts for their activities.

Require multi-factor authentication for all access to development endpoints and development resources.

Provide dedicated development endpoints on non-production networks for performing all development-related tasks. Provide separate endpoints on production networks for all other tasks.

Configure each development endpoint following zero trust principles.

A.2 Protect the Software

A.2.1 Protect All Forms of Code from Unauthorized Access and Tampering

A.2.1.1 Securely store code

Store all source code and configuration-as-code in a code repository, and restrict access to it based on the nature of the code. For example, opensource code intended for public access may need its integrity and availability protected; other code may also need its confidentiality protected.

Use version control features of the repository to track all changes made to the code with accountability to the individual account.

Use commit signing for code repositories.

Have the code owner review and approve all changes made to the code by others.

Use code signing to help protect the integrity of executables.

Use cryptography (e.g., cryptographic hashes) to help protect file integrity.

A.2.2 Provide a Mechanism for Verifying Software Release Integrity

A.2.2.1 Share software integrity verification information

Post cryptographic hashes for release files on a well-secured website.

Use an established certificate authority for code signing so that consumers' operating systems or other tools and services can confirm the validity of signatures before use.

Periodically review the code signing processes, including certificate renewal, rotation, revocation, and protection.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.2.3 Archive and Protect Each Software Release

A.2.3.1 Retain software releases

Store the release files, associated images, etc. in repositories following the organization's established policy. Allow read-only access to them by necessary personnel and no access by anyone else.

Store and protect release integrity verification information and provenance data, such as by keeping it in a separate location from the release files or by signing the data.

A.2.3.2 Share software provenance data

Make the provenance data available to software acquirers in accordance with the organization's policies, preferably using standards-based formats.

Make the provenance data available to the organization's operations and response teams to aid them in mitigating software vulnerabilities.

Protect the integrity of provenance data, and provide a way for recipients to verify provenance data integrity.

Update the provenance data every time any of the software's components are updated.

A.3 Produce Well-Secured Software

A.3.1 Design Software to Meet Security Requirements and Mitigate Security Risks

A.3.1.1 Model the security risk

Train the development team (security champions, in particular) or collaborate with a risk modeling expert to create models and analyze how to use a risk-based approach to communicate the risks and determine how to address them, including implementing mitigations.

Perform more rigorous assessments for high-risk areas, such as protecting sensitive data and safeguarding identification, authentication, and access control, including credential management.

Review vulnerability reports and statistics for previous software to inform the security risk assessment.

Use data classification methods to identify and characterize each type of data that the software will interact with.

A.3.1.2 Track design decisions

Record the response to each risk, including how mitigations are to be achieved and what the rationales are for any approved exceptions to the security requirements. Add any mitigations to the software's security requirements.

Maintain records of design decisions, risk responses, and approved exceptions that can be used for auditing and maintenance purposes throughout the rest of the software life cycle.

Periodically re-evaluate all approved exceptions to the security requirements, and implement changes as needed.

A.3.1.3 Use standardized security features and services

Maintain one or more software repositories of modules for supporting standardized security features and services. "

Determine secure configurations for modules for supporting standardized security features and services, and make these configurations available (e.g., as configuration-as-code) so developers can readily use them.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Define criteria for which security features and services must be supported by software to be developed.

A.3.2 Review the Software Design to Verify Compliance with Security Requirements and Risk Information

A.3.2.1 Validate the software design

Review the software design to confirm that it addresses applicable security requirements.

Review the risk models created during software design to determine if they appear to adequately identify the risks.

Review the software design to confirm that it satisfactorily addresses the risks identified by the risk models.

Have the software's designer correct failures to meet the requirements.

Change the design and/or the risk response strategy if the security requirements cannot be met.

Record the findings of design reviews to serve as artifacts (e.g., in the software specification, in the issue tracking system, in the threat model).

A.3.3 Reuse Existing, Well-Secured Software When Feasible Instead of Duplicating Functionality

A.3.3.1 Maintain well secured third-party software components

Review and evaluate third-party software components in the context of their expected use. If a component is to be used in a substantially different way in the future, perform the review and evaluation again with that new context in mind.

Determine secure configurations for software components, and make these available (e.g., as configuration-as-code) so developers can readily use the configurations.

Obtain provenance information (e.g., SBOM, source composition analysis, binary software composition analysis) for each software component, and analyze that information to better assess the risk that the component may introduce.

Establish one or more software repositories to host sanctioned and vetted open-source components.

Maintain a list of organization-approved commercial software components and component versions along with their provenance data.

Designate which components must be included in software to be developed.

Implement processes to update deployed software components to newer versions, and retain older versions of software components until all transitions from those versions have been completed successfully.

If the integrity or provenance of acquired binaries cannot be confirmed, build binaries from source code after verifying the source code's integrity and provenance.

A.3.3.2 Create and maintain well secured in-house software components

Follow organization-established security practices for secure software development when creating and maintaining the components.

Determine secure configurations for software components, and make these available (e.g., as configuration-as-code) so developers can readily use the configurations.

Maintain one or more software repositories for these components.

Designate which components must be included in software to be developed.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Implement processes to update deployed software components to newer versions, and maintain older versions of software components until all transitions from those versions have been completed successfully.

A.3.3.3 Verify security of third party software components

Regularly check whether there are publicly known vulnerabilities in the software modules and services that vendors have not yet fixed.

Build into the toolchain automatic detection of known vulnerabilities in software components.

Use existing results from commercial services for vetting the software modules and services.

Ensure that each software component is still actively maintained and has not reached end of life; this should include new vulnerabilities found in the software being remediated.

Determine a plan of action for each software component that is no longer being maintained or will not be available in the near future.

Confirm the integrity of software components through digital signatures or other mechanisms.

Review, analyze, and/or test code.

A.3.4 Create Source Code by Adhering to Secure Coding Practices

A.3.4.1 Follow appropriate secure coding practices

Validate all inputs, and validate and properly encode all outputs.

Avoid using unsafe functions and calls.

Detect errors, and handle them gracefully.

Provide logging and tracing capabilities.

Use development environments with automated features that encourage or require the use of secure coding practices with just-in-time training in-place.

Follow procedures for manually ensuring compliance with secure coding practices when automated methods are insufficient or unavailable.

Use tools (e.g., linters, formatters) to standardize the style and formatting of the source code.

Check for other vulnerabilities that are common to the development languages and environment.

Have the developer review their own human-readable code to complement (not replace) code review performed by other people or tools.

A.3.5 Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security

A.3.5.1 Choose compiler, interpreter and build tools

Use up-to-date versions of compiler, interpreter, and build tools.

Follow change management processes when deploying or updating compiler, interpreter, and build tools, and audit all unexpected changes to tools.

Regularly validate the authenticity and integrity of compiler, interpreter, and build tools.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.3.5.2 Configure compiler, interpreter and build tools

Enable compiler features that produce warnings for poorly secured code during the compilation process.

Implement the "clean build" concept, where all compiler warnings are treated as errors and eliminated except those determined to be false positives or irrelevant.

Perform all builds in a dedicated, highly controlled build environment.

Enable compiler features that randomize or obfuscate execution characteristics, such as memory location usage, that would otherwise be predictable and thus potentially exploitable.

Test to ensure that the features are working as expected and are not inadvertently causing any operational issues or other problems.

Continuously verify that the approved configurations are being used.

Make the approved tool configurations available as configuration-as-code so developers can readily use them.

A.3.6 Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements

A.3.6.1 Determine when to review or analyze code

Follow the organization's policies or guidelines for when code review should be performed and how it should be conducted. This may include third party code and reusable code modules written in-house.

Follow the organization's policies or guidelines for when code analysis should be performed and how it should be conducted.

Choose code review and/or analysis methods based on the stage of the software.

A.3.6.2 Review and analyze code

Perform peer review of code, and review any existing code review, analysis, or testing results as part of the peer review.

Use expert reviewers to check code for backdoors and other malicious content.

Use peer reviewing tools that facilitate the peer review process, and document all discussions and other feedback.

Use a static analysis tool to automatically check code for vulnerabilities and compliance with the organization's secure coding standards with a human reviewing the issues reported by the tool and remediating them as necessary.

Use review checklists to verify that the code complies with the requirements.

Use automated tools to identify and remediate documented and verified unsafe software practices on a continuous basis as human-readable code is checked into the code repository.

Identify and document the root causes of discovered issues.

Document lessons learned from code review and analysis in a wiki that developers can access and search.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.3.7 Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements

A.3.7.1 Determine when and how to test code

Follow the organization's policies or guidelines for when code testing should be performed and how it should be conducted (e.g., within a sandboxed environment). This may include third-party executable code and reusable executable code modules written in-house.

Choose testing methods based on the stage of the software.

A.3.7.2 Test code

Perform robust functional testing of security features.

Integrate dynamic vulnerability testing into the project's automated test suite.

Incorporate tests for previously reported vulnerabilities into the project's test suite to ensure that errors are not reintroduced.

Take into consideration the infrastructures and technology stacks that the software will be used with in production when developing test plans.

Use fuzz testing tools to find issues with input handling.

If resources are available, use penetration testing to simulate how an attacker might attempt to compromise the software in high-risk scenarios.

Identify and record the root causes of discovered issues.

Document lessons learned from code testing in a wiki that developers can access and search.

Use source code, design records, and other resources when developing test plans.

A.3.8 Configure Software to Have Secure Settings by Default

A.3.8.1 Define a secure baseline

Conduct testing to ensure that the settings, including the default settings, are working as expected and are not inadvertently causing any security weaknesses, operational issues, or other problems.

A.3.8.2 Implement and document the secure baseline

Verify that the approved configuration is in place for the software.

Document each setting's purpose, options, default value, security relevance, potential operational impact, and relationships with other settings.

Use authoritative programmatic technical mechanisms to record how each setting can be implemented and assessed by software administrators.

Store the default configuration in a usable format and follow change control practices for modifying it (e.g., configuration-as-code).

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

A.4 Respond to Vulnerabilities

A.4.1 Identify and Confirm Vulnerabilities on an Ongoing Basis

A.4.1.1 Investigate vulnerability reports

Monitor vulnerability databases, security mailing lists, and other sources of vulnerability reports through manual or automated means.

Use threat intelligence sources to better understand how vulnerabilities in general are being exploited.

Automatically review provenance and software composition data for all software components to identify any new vulnerabilities they have.

A.4.1.2 Evaluate software for vulnerabilities

Configure the toolchain to perform automated code analysis and testing on a regular or continuous basis for all supported releases.

A.4.1.3 Implement vulnerability disclosure and remediation

Establish a vulnerability disclosure program, and make it easy for security researchers to learn about your program and report possible vulnerabilities.

Have a Product Security Incident Response Team (PSIRT) and processes in place to handle the responses to vulnerability reports and incidents, including communications plans for all stakeholders.

Have a security response playbook to handle a generic reported vulnerability, a report of zero-days, a vulnerability being exploited in the wild, and a major ongoing incident involving multiple parties and open-source software components.

Periodically conduct exercises of the product security incident response processes.

A.4.2 Assess, Prioritize, and Remediate Vulnerabilities

A.4.2.1 Analyze vulnerabilities

Use existing issue tracking software to record each vulnerability.

Perform risk calculations for each vulnerability based on estimates of its exploitability, the potential impact if exploited, and any other relevant characteristics.

A.4.2.2 Implement risk responses

Make a risk-based decision as to whether each vulnerability will be remediated or if the risk will be addressed through other means (e.g., risk acceptance, risk transference), and prioritize any actions to be taken.

If a permanent mitigation for a vulnerability is not yet available, determine how the vulnerability can be temporarily mitigated until the permanent solution is available, and add that temporary remediation to the plan.

Develop and release security advisories that provide the necessary information to software acquirers, including descriptions of what the vulnerabilities are, how to find instances of the vulnerable software, and how to address them (e.g., where to get patches and what the patches change in the software; what configuration settings may need to be changed; how temporary workarounds could be implemented).

Deliver remediations to acquirers via an automated and trusted delivery mechanism. A single remediation could address multiple vulnerabilities.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.

Update records of design decisions, risk responses, and approved exceptions as needed.

A.4.3 Analyze Vulnerabilities to Identify Their Root Causes

A.4.3.1 Determine root causes

Record the root cause of discovered issues.

Record lessons learned through root cause analysis in a wiki that developers can access and search.

A.4.3.2 Identify patterns with root causes

Record lessons learned through root cause analysis in a wiki that developers can access and search.

Add mechanisms to the toolchain to automatically detect future instances of the root cause.

Update manual processes to detect future instances of the root cause.

A.4.3.3 Eradicate similar vulnerabilities

None

A.4.3.4 Update the SDLC process

Record lessons learned through root cause analysis in a wiki that developers can access and search.

Plan and implement changes to the appropriate SDLC practices.

Copyright by INCITS. Licensed to IT Help San Diego Inc. user. ANSI store order # X_1179149. Downloaded 03/19/2026. Single user license only. Copying and networking prohibited.